

MDF → MySQL Migration Tool

MSSQL → MySQL Übersetzungsplan

Implementierungslücken · Priorisierung · Umsetzungsplanung

#	Abschnitt	Inhalt
1	Statusübersicht	Welche Punkte sind bereits implementiert?
2	Fehlende Features	Detail-Analyse der 5 offenen Lücken
3	Implementierungsplan	Reihenfolge, Aufwand, Datei-Zuordnung
4	Akzeptanzkriterien	Wie wird Erfolg gemessen?

Kontext: Dieses Dokument beschreibt die noch fehlenden Übersetzungs-Regeln im Tool *mdf-to-mysql-migration*, die für eine saubere Migration der *Cockpit_Databases*-MDF nach MySQL/MariaDB erforderlich sind. Grundlage: MySQL_Migration_ToDo.md + MySQL_Portability_Report.md (Projekt Cockpit_ElectroPlating_2.0_Database).

1 · Statusübersicht

Stand: Branch **master** · geprüfte Dateien: **src/transform.py**, **src/deploy.py**, **src/mssql.py**.

#	Feature	Prio	Status
T1a	Typ-Mapping (alle außer TINYINT)	P1	■ implementiert
T1b	TINYINT → TINYINT UNSIGNED	P1	■ fehlt
T2	IDENTITY → AUTO_INCREMENT	P1	■ implementiert
T3	Defaults (GETDATE → CURRENT_TS)	P1	■ implementiert
T4	Storage-Optionen stripben	P1	■ implementiert
T5a	[name] → Backtick	P1	■ implementiert
T5b	N'...' Unicode-Literale in Views	P1	■ fehlt
T5c	GO / SET NOCOUNT ON	P1	n/a (MDF-Workflow)
V1	STRING_AGG → GROUP_CONCAT	P2	■ implementiert
V2	ISNULL → IFNULL	P2	■ implementiert
V3	CAST AS MONEY → DECIMAL	P2	■ implementiert
V4	OUTER/CROSS APPLY → LEFT JOIN	P2	■ implementiert
V5	CTE Versions-Hinweis	P2	■ fehlt
X1	MERGE → Flagging	P3	■ fehlt
X2	Trigger → Flagging + Hinweise	P3	■ fehlt
X3	OPENROWSET → Flagging	P3	n/a (MDF-Workflow)
X4	SQLCMD-Spezifika	P3	n/a (MDF-Workflow)

#	Feature	Prio	Status
P4	Engine-spez. Objekte → Flagging	P4	■ fehlt

Ergebnis P1/P2: Tabellen-Layer und View-Konvertierung sind zu ~85 % implementiert. Die 5 offenen Punkte (T1b, T5b, V5, X2, P4) blockieren einzelne Objekte, nicht die gesamte Migration.

2 · Fehlende Features – Detail-Analyse

T1b · TINYINT → TINYINT UNSIGNED

Problem: SQL Server's TINYINT hat den Wertebereich 0–255 (unsigned). MySQL's TINYINT ist standardmäßig signed (–128 bis 127). Werte > 127 aus der MDF würden beim INSERT in MySQL überlaufen.

Vorher (MDF)	Nachher (MySQL)
[Layer] TINYINT NOT NULL`	`` `Layer` TINYINT UNSIGNED NOT NULL ``

Betroffene Datei: src/transform.py — TYPE_MAP

Fix: "tinyint": "TINYINT UNSIGNED" in TYPE_MAP ändern.

Aufwand: 1 Zeile + 1 Test.

T5b · N'...' Unicode-Literale in View-Körpern

Problem: SQL Server verwendet `N'text` für Unicode-String-Literale. In MySQL sind alle Strings standardmäßig Unicode (UTF-8) – das `N`-Präfix wird nicht unterstützt und führt zu Syntax-Fehlern.

Vorher (T-SQL in View)	Nachher (MySQL)
WHERE Name = N'Test'	WHERE Name = 'Test'
CONVERT(NVARCHAR, N'abc')	CONVERT(CHAR, 'abc')

Betroffene Datei: src/transform.py — convert_view_sql()

Fix: Regex `r'\bN'` → `''` nach den bestehenden Typ-Ersetzungen.

Aufwand: 1 Zeile + 1 Test.

V5 · CTE Versions-Hinweis

Problem: Mehrere Views verwenden CTEs (`WITH ... AS (...)`). Diese sind nur ab MySQL 8.0 / MariaDB 10.2 unterstützt. Aktuell erscheint kein Hinweis im generierten DDL.

Gewünschter Output im DDL
-- ■ CTE verwendet: erfordert MySQL >= 8.0 / MariaDB >= 10.2
WITH LastValues AS (SELECT ...)

Betroffene Datei: src/transform.py — convert_view_sql()

Fix: Nach Header-Entfernung: `if re.search(r'\bWITH\b.\*\bAS\b.\*\(', sql, re.IGNORECASE|re.DOTALL):` → `warnings.append('CTE verwendet: ...')`

Aufwand: 3 Zeilen + 1 Test.

X2 · Trigger-Erkennung & MANUELL-Flagging

Problem: SQL Server-Trigger sind set-basiert ('inserted'/'deleted'-Pseudotabellen) und können nicht 1:1 in MySQL's row-basierte Trigger übersetzt werden. Aktuell werden Trigger aus der MDF gelesen und unverarbeitet ins DDL geschrieben – dies führt zu Syntax-Fehlern auf MySQL.

■ Betroffen: **alle 12 LastChange-Trigger** + Audit-Trigger (TriggerTableAudit). Diese erscheinen im generierten DDL als ungültige T-SQL-Blöcke.

Was das Tool tun soll	Ausgabe im DDL
Trigger-DDL als MANUELL flaggen	-- ■ MANUELL: CREATE TRIGGER trg_TableX_LastChange -- Empfehlung: Spalte LastChange DATETIME -- DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP -- (Trigger-Body wird NICHT übersetzt)
Mapping-Hinweis für Audit-Trigger	-- inserted → NEW deleted → OLD -- IF UPDATE(col) → NEW.col <=> OLD.col -- Gerüst: CREATE TRIGGER ... FOR EACH ROW BEGIN...END

Betroffene Datei: src/transform.py — generate_mysql_ddl() + neuer Block
MDF-Lese-Seite: src/mssql.py — Trigger aus sys.triggers lesen
Aufwand: ~2–3 Stunden (Lesen + Flagging + Hinweis-Generator + Tests).

P4 · Engine-spezifische Objekte – Flagging-Mechanismus

Problem: LastChange-Trigger, Audit-Generator und ViewAuditChanges sind komplett engine-spezifisch und müssen für MySQL **neu geschrieben** werden. Das Tool sollte diese Objekte **identifizieren** und einen separaten Report ausgeben – statt sie still zu überspringen.

Objekt-Typ	Anzahl	Empfehlung
LastChange-Trigger (trg_*_LastChange)	12	Ersetzen durch ON UPDATE CURRENT_TIMESTAMP-Spalte
Audit-Trigger (TriggerTableAudit)	1	Neu schreiben: FOR EACH ROW + JSON_OBJECT(...)
ViewAuditChanges	1	Entfällt – row-Trigger liefert direkt nur Änderungen

Betroffene Datei: src/transform.py — generate_mysql_ddl(), neuer Abschnitt
Output: Liste 'MANUELL ZU PRUEFEN' am Ende des generierten DDL
Aufwand: ~1 Stunde (Erkennung + Report-Block).

3 · Implementierungsplan

Alle 5 offenen Punkte sind unabhängig voneinander umsetzbar. Empfohlene Reihenfolge nach Aufwand/Nutzen-Verhältnis:

	Beschreibung	Datei(en)	Aufwand	Status
1b	TINYINT UNSIGNED	src/transform.py TYPE_MAP	1 Zeile + 1 Test	■ Sofort
5b	N'...' entfernen	src/transform.py convert_view_sql()	1 Zeile + 1 Test	■ Sofort
5	CTE-Hinweis	src/transform.py convert_view_sql()	3 Zeilen + 1 Test	■ Sofort
2	Trigger-Flagging + Hinweise	src/mssql.py src/transform.py	2–3 h + Tests	■ Issue #
4	Engine-spez. Flagging-Report	src/transform.py generate_mysql_ddl()	~1 h + Tests	■ Issue #

Quick Wins (Punkte 1–3): T1b, T5b und V5 können in einem einzigen kleinen Commit umgesetzt werden – insgesamt unter 10 Zeilen Code, sofortiger Nutzen für alle View-Migrationen.

Punkte 4–5 (Trigger/Engine-Flagging) erfordern zunächst das Lesen von Trigger-DDL aus der MDF (sys.triggers + sys.sql_modules). Empfehlung: als separate Issues im Repo anlegen.

3.1 · Konkrete Code-Änderungen (Quick Wins)

T1b – TYPE_MAP ändern

```
# src/transform.py – TYPE_MAP
# Vorher:
"tinyint": "TINYINT",
# Nachher:
"tinyint": "TINYINT UNSIGNED",
```

T5b – N-String-Literale in convert_view_sql()

```
# src/transform.py – convert_view_sql(), nach den Typ-Ersetzungen
# N'...' Unicode-Literale entfernen (MySQL: alle Strings sind UTF-8)
sql = re.sub(r"\bN(?:=')", '', sql)
```

V5 – CTE-Versions-Hinweis in convert_view_sql()

```
# src/transform.py – convert_view_sql(), nach Header-Entfernung
if re.search(r'\bWITH\b', sql, re.IGNORECASE):
    warnings.append(
        'CTE verwendet (WITH ...): erfordert MySQL >= 8.0 / MariaDB >= 10.2'
    )
```

4 · Akzeptanzkriterien

Erfolg wird auf zwei Ebenen gemessen: Unit-Tests im Repo und Deploy-Test gegen eine leere MariaDB-Instanz.

ID	Punkt	Test-Kriterium	Test-Datei
T1b	TINYINT	TYPE_MAP['tinyint'] == 'TINYINT UNSIGNED'	test_transform.py
T5b	N-String	convert_view_sql("WHERE x = N'abc'") → keine N' im Output	test_transform.py
V5	CTE-Hint	convert_view_sql('WITH cte AS (...) SELECT ...') → warnings enthält 'CTE'	test_transform.py
X2	Trigger	generate_mysql_ddl() mit Trigger-DDL → Kommentar '-- ■ MANUELL' im Output	test_transform.py test_mssql.py
P4	Engine-Flag	Flagging-Report am Ende des DDL enthält LastChange-Trigger-Namen	test_transform.py
Alle	Smoke-Test	Generiertes DDL deploybar auf MariaDB 10.6 ohne Fehler (py -m pytest + manueller Deploy-Test)	tests/ + MariaDB

Aktueller Test-Stand: 189 Tests, alle grün (Branch master). Neue Tests für T1b, T5b, V5 können als Ergänzung zu den bestehenden test_transform.py-Tests hinzugefügt werden.